ACTIVIDAD I

Aguilar Andrade Ángel de Jesús, Rodríguez Cruz Ángel Emiliano
Universidad Nacional Autónoma de México, Facultad de Estudios Superiores Cuautitlán

10 de marzo de 2026

Resumen

El presente documento detalla el desarrollo, implementación y análisis de un sistema de generación y ordenamiento de datos escrito íntegramente en lenguaje ensamblador para el procesador AVR del microcontrolador ATmega328P. Se programó un algoritmo para depositar 100 números pseudoaleatorios en la memoria SRAM del dispositivo. Posteriormente, se implementaron los algoritmos de ordenamiento de Burbuja (Bubble Sort) y Selección (Selection Sort) interactuando directamente con los módulos de hardware del CPU. Finalmente, se utilizó el depurador de Microchip Studio para medir el tiempo de ejecución exacto en ciclos de reloj, comparando el rendimiento empírico del hardware con el modelo teórico de complejidad computacional expresado mediante la notación Big-O.

Introducción

Los microcontroladores son sistemas embebidos fundamentales en la ingeniería moderna. Su programación a bajo nivel mediante lenguaje ensamblador permite un control total sobre los recursos de hardware, optimizando el uso de la memoria, el tiempo de ejecución y el consumo energético. Dentro de las ciencias de la computación, el ordenamiento de datos es un proceso crítico que sirve como base para algoritmos de búsqueda, análisis estadístico y procesamiento de señales.

En el presente trabajo, se aborda la escritura de programas en ensamblador para manipular directamente los registros del CPU y la memoria SRAM del microcontrolador ATmega328P. El objetivo principal es generar una lista de 100 números pseudoaleatorios y ordenarlos utilizando dos métodos clásicos: Burbuja y Selección, evaluando su eficiencia teórica y práctica.

Marco Teorico.

1. Algoritmos de Ordenamiento Los algoritmos de ordenamiento son secuencias de instrucciones diseñadas para reorganizar una matriz bajo una relación de orden específica. Para este trabajo, se eligieron dos algoritmos fundamentales:

- **Bubble Sort (Burbuja):** Evalúa la matriz comparando elementos adyacentes y permutándolos si se encuentran en el orden incorrecto. Este proceso se repite hasta que no se requieren más intercambios.
- **Selection Sort (Selección):** Divide el arreglo en una sublista ordenada y otra desordenada. Itera sobre la parte desordenada buscando el valor mínimo absoluto y lo intercambia con el primer elemento desordenado disponible.

2. Notación Big-O La notación asintótica Big-O es una herramienta matemática en ciencias de la computación utilizada para describir el rendimiento de un algoritmo. Permite proyectar cómo escalará el tiempo de ejecución a medida que el tamaño de los datos de entrada (n) tiende a infinito. A continuación, se muestra la complejidad de varios algoritmos comunes:

- Burbuja: Mejor Caso $O(n)$ | Promedio $O(n^2)$ | Peor Caso $O(n^2)$
- Selección: Mejor Caso $O(n^2)$ | Promedio $O(n^2)$ | Peor Caso $O(n^2)$
- Inserción: Mejor Caso $O(n)$ | Promedio $O(n^2)$ | Peor Caso $O(n^2)$
- Merge Sort: Mejor Caso $O(n \log n)$ | Promedio $O(n \log n)$ | Peor Caso $O(n \log n)$
- Quick Sort: Mejor Caso $O(n \log n)$ | Promedio $O(n \log n)$ | Peor Caso $O(n^2)$

Desarrollo e Implementación

Para la implementación se utilizó el entorno Microchip Studio, generando el código fuente en ensamblador.

Algoritmo 1: Generación de números aleatorios (LCG)

Entrada: Semilla inicial (19)

Salida: Arreglo de 100 números en SRAM

```
inicio
contador = 100
mientras contador > 0 hacer
semilla = (semilla * 13) + 7
guardar semilla en tabla de RAM
avanzar puntero
contador = contador - 1
fin mientras
fin
```

Algoritmo 2: Ordenamiento de Burbuja

Entrada: Tabla de 100 números desordenados

Salida: Tabla ordenada

```
inicio
n = 100
repetir
hubo_intercambio = falso
para i = 0 hasta n-1 hacer
si tabla[i] > tabla[i+1] entonces
intercambiar(tabla[i], tabla[i+1])
```

8 hubo_intercambio = verdadero

9 fin si 10 fin para

11 n = n - 1

12 hasta que hubo_intercambio == falso

13 fin

Algoritmo 3: Ordenamiento por Selección

Entrada: Tabla de 100 números desordenados

Salida: Tabla ordenada

1 inicio

2 n = 100

3 para i = 0 hasta n-1 hacer

4 indice_minimo = i

5 para j = i+1 hasta n hacer

6 si tabla[j] < tabla[indice_minimo] entonces

7 indice_minimo = j

8 fin si

9 fin para

10 si indice_minimo != i entonces

11 intercambiar(tabla[i], tabla[indice_minimo])

12 fin si

13 fin para

14 fin

Análisis de Resultados

La ejecución y comprobación del sistema se llevó a cabo utilizando la herramienta Stopwatch del depurador interno de Microchip Studio para determinar el tiempo que tarda el algoritmo 1 y 2 en realizar el ordenamiento de los 100 números.

BUBLLE

Processor Status

Name	Value
Program Counter	0x00000008
Stack Pointer	0x0800
X Register	0x0164
Y Register	0x022C
Z Register	0x016E
Status Register	I T H S V N Z C
Cycle Counter	71502
Frequency	16.000 MHz
Stop Watch	4,322.88 µs
Registers	
R00	0x56
R01	0x02
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00

SELECTION

Processor Status

Name	Value
Program Counter	0x00000009
Stack Pointer	0x0800
X Register	0x022C
Y Register	0x022B
Z Register	0x022B
Status Register	I T H S V N Z
Cycle Counter	123790
Frequency	16.000 MHz
Stop Watch	3,268.00 µs
Registers	
R00	0x56
R01	0x02
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00

Basado en la teoría, los algoritmos de Burbuja y Selección poseen la misma complejidad de $O(n^2)$ para un caso promedio. Sin embargo, el análisis a nivel de microarquitectura reveló una marcada diferencia en los ciclos de reloj consumidos por el CPU.
En la arquitectura AVR, las operaciones de memoria (instrucciones de carga y almacenamiento) requieren más ciclos de reloj que las operaciones en los registros de trabajo. El algoritmo de Selección resultó más eficiente en tiempo absoluto, puesto que minimiza los accesos a la memoria SRAM. Mientras que el método de Burbuja ejecuta constantes operaciones de intercambio en memoria, Selección restringe los intercambios a operaciones de escritura mínimas, confinando la búsqueda del elemento menor exclusivamente a lecturas rápidas en los registros.

Conclusión

El desarrollo del presente proyecto permitió consolidar el manejo de la arquitectura interna del microcontrolador ATmega328P. Se evidenció que, si bien la notación asintótica Big-O es imprescindible para clasificar algoritmos, la implementación en sistemas embebidos demanda un análisis riguroso de hardware. El costo en ciclos de reloj de las operaciones hacia la memoria RAM penaliza los tiempos de ejecución, demostrando que optimizar el uso de los registros de la ALU es imperativo para maximizar el rendimiento.

Bibliografia

[1] "Sorting Algorithms Explained with Examples in JavaScript, Python, Java, and C++," freeCodeCamp.org. Accessed: Mar. 03, 2026. [Online]. Available: https://www.freecodecamp.org/news/sorting-algorithms-explained-with-examples-in-python-java-and-c/
[2] "Big O Notation Explained with Examples," freeCodeCamp.org. Accessed: Mar. 03, 2026. [Online]. Available: https://www.freecodecamp.org/news/big-o-notation-explained-with-examples/
[3] "Algoritmos de ordenaci ʃn explicados con ejemplos en JavaScript, Python, Java y C++," freeCodeCamp.org. Accessed: Mar. 03, 2026. [Online]. Available: https://www.freecodecamp.org/espanol/news/algoritmos-de-ordenacion-explicados-con-ejemplos-en-javascript-python-java-y-c/
[4] "Understanding the Role of an Assembler in Computer Architecture," DEV Community. Accessed: Mar. 03, 2026. [Online]. Available: https://dev.to/adityabhuyan/understanding-the-role-of-an-assembler-in-computer-architecture-2hni